

Test Unitaire n°1

ID du test	TU01
Nom du module	API d'insertion de données
Fonction / Classe testée	Enregistrement Temp, Hum, Porte
Fichier Source	post_data.php
Auteur	Rémy QUELIN
Date	02/05/2026
Framwork utilisé	Debian

Description du Test

Objectif	Vérifier que les trois paramètres envoyés par URL sont Correctement insérés dans la base Maria DB.
Type de teste	Test de validation des données et d'insertion SQL.

Préconditions du Test

État du système : Le serveur Apache et la base Maria DB doivent être actives.

Données d'entrée

- Temp (Float) : 25.5
- Hum (Float) : 40.0
- Door (Int) : 0

Étapes d'exécution

Étape 1	Sur la Raspberry appeler l'URL via une commande curl avec les 3 paramètres.
Étape 2	Vérifier le message de retour du script.
Étape 3	Se connecter à la base de données Maria DB.
Étape 4	Exécuter <code>Use securite_local ;</code>
Étape 5	Exécuter <code>SELECT * FROM sensors_data ;</code>

Résultat attendu

Valeur attendue	Message "Succès : Données insérées dans Maria DB "
Comportement attendu	Trois nouvelles écritures dans la BDD avec les valeurs insérée par la commande.

Résultat Obtenu

Valeur obtenue	Valeur insérée par la commande curl
Statut	OK
Commentaires	Les données sont bien insérées dans la BDD

Test Unitaire n°2

ID du test	TU02
Nom du module	API de récupération et supervision
Fonction / Classe testée	Détection des dépassements des seuils
Fichier Source	get_data.php
Auteur	Rémy QUELIN
Date	02/05/2026
Framwork utilisé	Debian

Description du test

L'objectif est de vérifier que le script identifie bien une valeur hors-norme (ex : humidité trop élevée) et l'ajoute au journal.

Préconditions

État du système : Serveur Apache actif et base Maria DB sécurité local accessible

Données d'entrée

Valeur stockée dans la base de données : Humidité = 80.0 (Seuil critique > 65%).

Étape d'exécution

Étape 1	Sur la Raspberry appeler l'URL via curl avec le paramètre : hum80
Étape 2	Vérifier le message de retour du script.
Étape 3	Mettre l'adresse IP du serveur de la page web dans un navigateur
Étape 4	Vérifier dans Journal d'Événement

Résultat attendu

Valeur attendue : La valeur 80.0 doit apparaître dans le journal d'évènement.

Résultat obtenu

Valeur Obtenue	L'humidité retournée est de 80%
Statut	La valeur est bien indiquée dans le Journal d'évènement
Commentaire	Les données sont correctement insérées dans la base de données

Test Unitaire n°3

ID du test	TU03
Nom du module	Programme d'acquisition local
Fonction / Classe testée	Lecture des entrées GPIO (Capteur de porte ILS)
Fichier Source	Main.cpp
Auteur	Rémy QUELIN
Date	12/05/2026
Framwork utilisé	Debian

Description du test

L'objectif est de vérifier que le programme C++ détecte instantanément le changement d'état physique du capteur magnétique (ouverture/fermeture de la porte).

Préconditions

La Raspberry Pi doit être sous tension, le capteur ILS branché sur les ports GPIO configurés. Le programme d'acquisition doit être lancé en mode "debug" dans le terminal.

Données d'entrée

Éloignement de l'aimant du capteur (simulant l'ouverture de la porte de la baie informatique).

Étape d'exécution

Étape 1	Lancer l'exécutable sur la Raspberry Pi (./acquisition).
Étape 2	Observer les logs initiaux
Étape 3	Éloigner manuellement l'aimant du capteur ILS
Étape 4	Observer la sortie console du programme
Étape 5	Rapprocher l'aimant pour simuler la fermeture
Étape 6	Observer la sortie console du programme

Résultat attendu

Le programme affiche immédiatement 'OUVERTE' dans la console lors de l'éloignement de l'aimant.

Résultat obtenu

Valeur Obtenue	Détection immédiate du changement d'état sur le port GPIO.
Statut	OK
Commentaire	La réactivité est conforme aux besoins de sécurité en temps réel.

Test Unitaire n°4

ID du test	TU04
Nom du module	Contrôle d'accès physique
Fonction / Classe testée	Pilotage du relais de gâche électrique
Fichier Source	Main.cpp
Auteur	Rémy QUELIN
Date	22/05/2026
Framwork utilisé	OS Embarqué Fanvil / Relais physique

Description du test

Vérifier qu'un badge RFID valide déclenche bien l'ouverture physique de la gâche électrique via le portier Fanvil connecté en mode rupture de courant (NC).

Préconditions

Le portier Fanvil (192.168.1.22) doit être alimenté sous tension (PoE). La gâche électrique doit être câblée sur la borne NC (Normally Closed) du relais. Un badge RFID valide doit être provisionné dans la mémoire du portier.

Données d'entrée

Lecture d'un badge RFID valide présenté devant le lecteur.

Étape d'exécution

Étape 1	Présenter le badge RFID valide devant la zone de lecture du portier Fanvil.
Étape 2	Écouter le signal sonore du portier et le déclenchement mécanique (clac) du relais interne.
Étape 3	Pousser manuellement la porte pour valider la rupture de maintien magnétique.

Résultat attendu

Le circuit NC s'ouvre, l'alimentation de la gâche est coupée, la gâche se libère et permet l'ouverture physique immédiate de la porte. Un signal sonore de validation avec basculement du relais temporisé à 10 secondes.

Résultat obtenu

Valeur Obtenue	Ouverture physique validée lors du déclenchement du relais.
Statut	OK

Test Unitaire n°5

ID du test	TU05
Nom du module	Serveur Vidéo de Vidéosurveillance
Fonction / Classe testée	Streaming Web HLS Basse Latence (hls.js)
Auteur	Rémy QUELIN
Date	12/05/2026
Framwork utilisé	Linux Systemd / FFmpeg / JavaScript

Description du test

L'objectif est valider la diffusion du flux vidéo de la caméra IP sur le Dashboard Web d'ASGA en limitant la latence à moins de 20 secondes grâce à la configuration agressive de la mémoire tampon.

Préconditions

La caméra IP et le NVR doivent diffuser le flux RTSP. Le service Linux ffmpeg.service doit tourner sur le conteneur Web et générer des segments de 1 seconde (.m3u8). Le Dashboard doit être ouvert dans le navigateur web client.

Données d'entrée

Mouvement physique direct devant l'objectif de la caméra IP.

Étape d'exécution

Étape 1	Se connecter à l'interface du Dashboard Web d'ASGA depuis un poste client.
Étape 2	Réaliser un mouvement rapide et identifiable (ex: passer la main) devant la caméra.
Étape 3	Mesurer le décalage temporel exact entre le geste physique et sa répercussion visuelle sur le lecteur HTML5 du Dashboard.

Résultat attendu

Le flux vidéo s'affiche avec un peu moins de 20 secondes de latence.

Résultat obtenu

Valeur Obtenue	Flux vidéo fluide affiché en temps réel avec une latence stable de ~10 seconde.
Statut	OK

Test Unitaire n°6

ID du test	TU06
Nom du module	Supervision et Gestion des Pannes Réseau
Fonction / Classe testée	Détection de perte de connectivité (Script <code>check_system.php</code>)
Fichier Source	<code>check_system.php</code>
Auteur	Rémy QUELIN
Date	12/05/2026
Framwork utilisé	Raspberry Pi OS / Architecture Docker

Description du test

L'objectif est de simuler une panne matérielle ou réseau courante en débranchant physiquement la Raspberry Pi (192.168.1.13) afin de vérifier que l'interface de supervision détecte l'anomalie et alerte le technicien en passant son état à "OFFLINE".

Préconditions

Le conteneur Web est actif. La Raspberry Pi est initialement démarrée et son statut affiche "ONLINE" sur le Dashboard.

Données d'entrée

Déconnexion physique de l'alimentation ou du câble Ethernet de la Raspberry Pi.

Étape d'exécution

Étape 1	Consulter le Dashboard et vérifier que la Raspberry Pi est bien marquée "ONLINE".
Étape 2	Débrancher physiquement la Raspberry Pi de la maquette.
Étape 3	Attendre l'exécution automatique en arrière-plan du script <code>check_system.php</code> (ou rafraîchir la page).
Étape 4	Observer le changement de couleur et de texte du voyant de l'équipement sur l'interface.

Résultat attendu

Le script `check_system.php` ne reçoit plus de réponse de la Raspberry Pi. Le Dashboard prend en compte cette erreur et bascule immédiatement le voyant de la Raspberry Pi de "ONLINE" (Vert) à "OFFLINE" (Rouge) pour signaler la panne.

Résultat obtenu

Valeur Obtenue	Perte de liaison détectée. Le Dashboard affiche instantanément le statut "OFFLINE" en rouge.
Statut	OK

